

Integral de camino de Feynman en mecánica cuántica

Monte Carlo cuántico con el algoritmo de Metropolis

José Francisco Argueta Valle

Escuela de Física, Facultad de Ciencias Naturales y Matemática
Universidad de El Salvador

Seminario de Física Computacional (FCL1109)

19 de junio de 2026

Contenido

- 1 Introducción
- 2 Teoría
- 3 Algoritmo y código
- 4 Resultados
- 5 Relación con la asignatura
- 6 Conclusiones

Motivación: otra forma de ver la mecánica cuántica

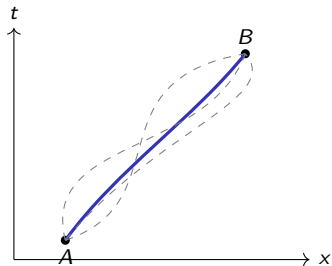
Feynman buscaba una formulación de la mecánica cuántica distinta de la imagen de Schrödinger, con una conexión más directa con la mecánica clásica. Dirac había sugerido el principio de mínima acción como el límite $\hbar \rightarrow 0$ de un principio cuántico de acción [1, 2].

Las dos imágenes

- **Schrödinger:** una ecuación diferencial para la función de onda $\psi(x, t)$ que se propaga en el tiempo.
- **Feynman:** la amplitud de ir de un punto a otro es una suma sobre todos los caminos del espacio-tiempo.

La idea central: suma sobre caminos

Contribuyen todos los caminos, cada uno con un peso $e^{iS/\hbar}$ que depende de su acción. El camino clásico es el de acción mínima y el más probable.



- Línea gruesa: trayectoria clásica $\bar{x}(t)$.
- Líneas discontinuas: otros caminos que muestrea la partícula.
- El peso $e^{iS/\hbar}$ asigna una fase a cada camino.

Propagador, acción e integral de camino

La función de onda se propaga mediante el propagador (función de Green) G [3]:

$$\psi(x_b, t_b) = \int dx_a G(x_b, t_b; x_a, t_a) \psi(x_a, t_a).$$

Feynman postuló que G es la suma sobre todos los caminos, cada uno ponderado por su acción $S = \int L dt$, con $L = T - V$ [2]:

$$G(b, a) = \sum_{\text{caminos}} e^{iS[b,a]/\hbar} \quad (\text{integral de camino}).$$

Límite clásico y tiempo imaginario

Como $\hbar \approx 10^{-34}$ Js, la acción es enorme ($S/\hbar \gtrsim 10^{20}$): las fases de caminos vecinos se cancelan salvo cerca del camino de acción mínima. Así emerge la trayectoria clásica.

El truco de cálculo es rotar a **tiempo imaginario** $t \rightarrow -i\tau$. La exponencial deja de oscilar y se vuelve un peso real e^{-S_E} . Para tiempos imaginarios grandes domina el estado de menor energía [4]:

$$|\psi_0(x)|^2 = \lim_{\tau \rightarrow \infty} e^{E_0 \tau} G(x, -i\tau; x, 0).$$

Discretización en una red de espacio-tiempo

Se coloca el camino en una red: posiciones x_i en cortes de tiempo imaginario separados ε . La acción euclidiana del oscilador armónico ($\hbar = m = \omega = 1$) es una suma sobre eslabones:

$$S_E = \sum_i \left[\frac{(x_{i+1} - x_i)^2}{2\varepsilon} + \frac{\varepsilon \omega^2}{2} x_i^2 \right] = \sum_i (\text{cinética} + \text{potencial}).$$

Resultado exacto de comparación

$$|\psi_0(x)|^2 = \frac{1}{\sqrt{\pi}} e^{-x^2}, \quad E_0 = \frac{1}{2} \hbar \omega, \quad \langle x^2 \rangle = \frac{1}{2}.$$

Qué queremos calcular y cómo

Objetivo: obtener la densidad del estado fundamental $|\psi_0(x)|^2$ y la energía E_0 del oscilador, sin resolver la ecuación de Schrödinger, evaluando la integral de muchas dimensiones del peso e^{-S_E} por Monte Carlo.

Estrategia (tres pasos):

- 1 Representar el camino como un arreglo de N posiciones x_i en una red.
- 2 Muestrear caminos con probabilidad $\propto e^{-S_E}$ usando Metropolis.
- 3 Acumular las posiciones visitadas en un histograma: ese histograma es $|\psi_0(x)|^2$, y de $\langle x^2 \rangle$ se obtiene E_0 .

Algoritmo de Metropolis

El peso e^{-S_E} se muestrea con el algoritmo de Metropolis, el mismo de la física estadística [5, 6]:

- ❶ Inicializar el camino (todos los eslabones en $x = 0$).
- ❷ Repetir muchas veces:
 - Elegir un eslabón i y proponer $x_i \rightarrow x_i + \delta$.
 - Calcular el cambio de acción ΔS .
 - Aceptar si $\Delta S \leq 0$, o si $e^{-\Delta S/\hbar} > \text{aleatorio}$.
 - Si no, rechazar y restaurar x_i .
 - Acumular la posición en el histograma de $|\psi_0|^2$.

Estructura del programa

El script `qmc_camino_feynman.py` se organiza en pocas funciones:

- `energia_total(camino)`: acción euclidiana completa (diagnóstico).
- `barrido_metropolis(...)`: actualiza una vez cada eslabón del camino.
- `simular(...)`: termaliza, mide y acumula el histograma y $\langle x^2 \rangle$.
- `densidad_analitica(x)`: la curva exacta de comparación.
- `main()`: ejecuta la simulación y genera las cuatro figuras.

Parámetros principales: `N_eslabones=200`, `eps=0.25`, `omega=1.0`, `barridos_medicion=40000`.

La acción discretizada en código

La función calcula la energía cinética (diferencias entre eslabones contiguos) y la potencial del oscilador, con frontera periódica vía `np.roll`:

```
def energia_total(camino, eps, omega, masa):  
    #  $x_{i+1} - x_i$  con frontera periódica  
    dx = np.roll(camino, -1) - camino  
    cinetica = 0.5 * masa * np.sum(dx**2) / eps  
    potencial = 0.5 * masa * omega**2 * eps * np.sum(camino**2)  
    return cinetica + potencial
```

En el muestreo no se recalcula toda la acción: solo cambia el aporte local del eslabón que se mueve, lo que hace cada paso muy barato.

Un barrido de Metropolis

Esquema rojo-negro: los eslabones que no son vecinos se actualizan a la vez, lo que vectoriza el barrido con numpy.

```
# Propuesta de desplazamiento para los eslabones de un color
x_nuevo = x_viejo + paso * (2*rng.random(idx.size) - 1)

# Acción local antes y después (solo términos que dependen de x_i)
s_viejo = ((x_viejo-vec_izq)**2 + (vec_der-x_viejo)**2)/(2*eps) \
          + 0.5*omega**2*eps*x_viejo**2
s_nuevo = ((x_nuevo-vec_izq)**2 + (vec_der-x_nuevo)**2)/(2*eps) \
          + 0.5*omega**2*eps*x_nuevo**2
delta_s = s_nuevo - s_viejo

# Criterio de Metropolis (aceptar o rechazar, vectorizado)
acepta = (delta_s <= 0) | (rng.random(idx.size) < np.exp(-delta_s/hbar))
camino[idx] = np.where(acepta, x_nuevo, x_viejo)
```

Medición: acumular el histograma

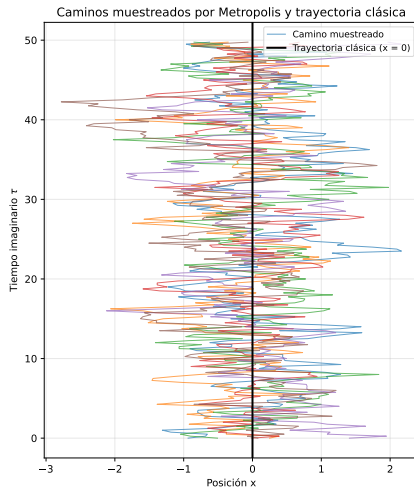
Tras termalizar, cada barrido aporta sus N posiciones al histograma y a la suma de x^2 . El histograma normalizado es la densidad $|\psi_0(x)|^2$:

```
for k in range(barridos_medicion):
    barrido_metropolis(camino, eps, omega, masa, paso, hbar, rng)
    conteo, _ = np.histogram(camino, bins=bordes_hist)
    histograma += conteo           # acumula  $|\psi_0(x)|^2$ 
    suma_x2 += np.sum(camino**2)  # para  $\langle x^2 \rangle$  y la energía

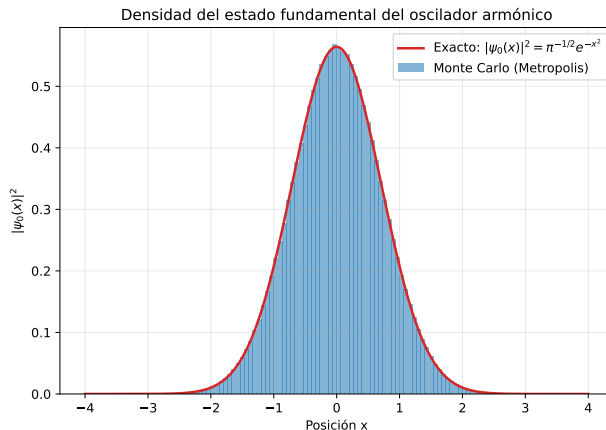
# normalizar para que el histograma integre a 1
densidad = histograma / (np.sum(histograma) * ancho)
```

La energía sale del virial: $E_0 = \omega^2 \langle x^2 \rangle$.

Caminos muestrados y trayectoria clásica

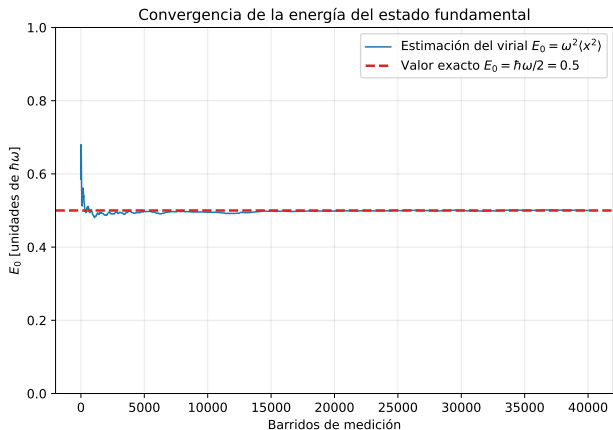


Función de onda del estado fundamental



El histograma de Monte Carlo reproduce la gaussiana exacta $\pi^{-1/2} e^{-x^2}$.

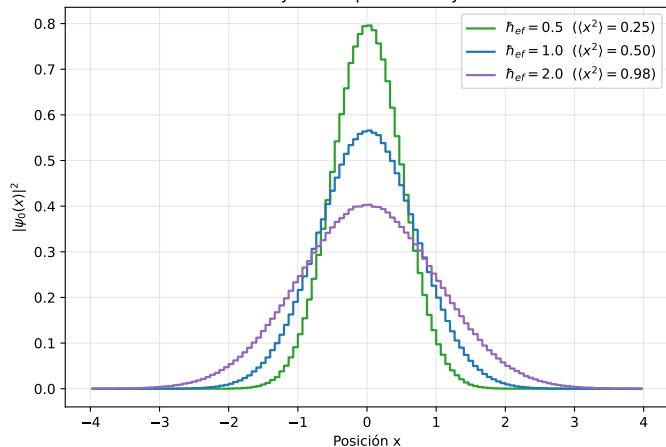
Convergencia de la energía



$E_0 = \omega^2 \langle x^2 \rangle$ converge a $\hbar\omega/2 = 0,5$ (se obtuvo 0,5001).

Efecto de $\hbar$ sobre las fluctuaciones

Efecto de $\hbar$: mayor $\hbar$ permite mayores fluctuaciones



Al aumentar $\hbar$ se permiten mayores fluctuaciones:

- la distribución se ensancha,
- $\langle x^2 \rangle$ crece como $\hbar/2$,
- en el límite $\hbar \rightarrow 0$ solo sobrevive el camino clásico.

Conexiones con los temas del curso

- **Metropolis y Monte Carlo** (termodinámica y física estadística, Cap. 7 de Landau): el mismo algoritmo que muestrea el modelo de Ising.
- **Ecuación de difusión** (Cap. 4): Schrödinger en tiempo imaginario es una ecuación de difusión, y por eso $e^{iS/\hbar}$ pasa a ser e^{-S_E} .
- **Oscilador armónico** (Cap. 3 clásico, Cap. 6 cuántico): el sistema de prueba ya estudiado en clase.
- **Números aleatorios y muestreo**, base de los métodos Monte Carlo.
- **Trayectoria clásica** como solución de las ecuaciones de movimiento (EDOs con rk4): es el límite $\hbar \rightarrow 0$ de la suma.

Conclusiones

- La integral de camino reformula la mecánica cuántica como suma sobre caminos y reproduce la misma física que la imagen de Schrödinger.
- En tiempo imaginario, el Monte Carlo con Metropolis recupera $|\psi_0(x)|^2 = \pi^{-1/2}e^{-x^2}$ y $E_0 = 0,5 \hbar\omega$ (se obtuvo 0,5001).
- El camino clásico domina; las fluctuaciones crecen con $\hbar$, lo que ilustra de forma directa el límite clásico.
- El método enlaza la mecánica cuántica con los métodos Monte Carlo y de ecuaciones en derivadas parciales vistos en la asignatura.

Referencias I

- [1] J. J. Sakurai. *Advanced Quantum Mechanics*. Reading, Massachusetts: Addison-Wesley, 1967.
- [2] Richard P. Feynman. «Space-Time Approach to Non-Relativistic Quantum Mechanics». En: *Reviews of Modern Physics* 20.2 (1948), págs. 367-387. DOI: 10.1103/RevModPhys.20.367.
- [3] Richard P. Feynman y Albert R. Hibbs. *Quantum Mechanics and Path Integrals*. New York: McGraw-Hill, 1965.
- [4] Rubin H. Landau, Manuel J. Páez y Cristian C. Bordeianu. *Computational Problems for Physics: With Guided Solutions Using Python*. Boca Raton: CRC Press, 2018. ISBN: 978-1138705418.
- [5] Nicholas Metropolis et al. «Equation of State Calculations by Fast Computing Machines». En: *The Journal of Chemical Physics* 21.6 (1953), págs. 1087-1092. DOI: 10.1063/1.1699114.

- [6] Michael Creutz y Barry Freedman. «A Statistical Approach to Quantum Mechanics». En: *Annals of Physics* 132.2 (1981), págs. 427-462. DOI: [10.1016/0003-4916\(81\)90074-9](https://doi.org/10.1016/0003-4916(81)90074-9).

Gracias por su atención

Preguntas y comentarios

Apéndice: código completo I

Archivo `codigo/qmc_camino_feynman.py` (adaptación del listado 6.24, QMC.py, de Landau y Páez).

"""

Integral de camino de Feynman por Monte Carlo cuántico (algoritmo de Metropolis).

Seminario de Física Computacional (FCO4101) - Universidad de El Salvador.

Tema: Feynman Path Integral Quantum Mechanics.

*Bibliografía principal: Landau & Páez, Computational Problems for Physics (2018),
sección 6.11 y listado 6.24 (programa QMC.py).*

Idea física

*La amplitud de propagación se escribe como una suma sobre todos los caminos
ponderados por la acción:*

$$G(b, a) = \text{sum_caminos} \exp(i S[b,a] / \hbar), \quad S = \text{integral de } L \, dt, \quad L = T - V.$$

*Pasando a tiempo imaginario ($t \rightarrow -i \tau$) la exponencial deja de oscilar y se
convierte en un peso de Boltzmann $\exp(-S_E)$. Para tiempos imaginarios grandes
el peso queda dominado por el estado fundamental, de modo que el histograma de
las posiciones visitadas reproduce la densidad de probabilidad del estado base:*

$$|\psi_0(x)|^2 = (1/Z) \lim_{\tau \rightarrow \infty} \text{integral } dx_1 \dots dx_{N-1} \exp(-\epsilon \tau E).$$

*Este script discretiza el tiempo imaginario en una red de N eslabones y usa el
algoritmo de Metropolis (sección 7.4 del libro) para muestrear los caminos.*

Apéndice: código completo II

*Sistema de prueba: oscilador armónico unidimensional en unidades naturales
($\hbar = m = \omega = 1$). El resultado exacto que sirve de comparación es*

$$|\psi_0(x)|^2 = (1/\sqrt{\pi}) \exp(-x^2), \quad E_0 = \hbar\omega/2 = 0.5, \\ \langle x^2 \rangle = \hbar/(2 m \omega) = 0.5.$$

Adaptación respecto al listado 6.24 del libro

- Sin VPython: se usa Matplotlib con backend no interactivo y se guardan PDFs.
- El muestreo de Metropolis se vectoriza con el esquema rojo-negro (se actualizan primero los eslabones pares y luego los impares), válido porque el acoplamiento solo une vecinos contiguos. Así cada barrido completo es una operación de numpy.
- Se implementa el muestreo aquí mismo (fcl1109.py no incluye Metropolis).

Autor: José Francisco Argueta Valle.

"""

```
import os
```

```
import numpy as np
```

```
import matplotlib
```

```
matplotlib.use("Agg") # backend sin ventana: solo se guardan archivos
```

```
import matplotlib.pyplot as plt
```

```
# -----
```

Apéndice: código completo III

```
# Parámetros físicos y numéricos
# -----
N_eslabones = 200      # número de cortes de tiempo imaginario (red periódica)
eps = 0.25             # paso de tiempo imaginario [adimensional]; beta = N*eps
omega = 1.0            # frecuencia del oscilador armónico [adimensional]
masa = 1.0             # masa de la partícula [adimensional]

paso_propuesta = 1.0   # amplitud máxima del desplazamiento propuesto en Metropolis
semilla = 2026         # semilla del generador aleatorio (reproducibilidad)

barridos_termalizacion = 5000  # barridos descartados hasta alcanzar equilibrio
barridos_medicion = 40000      # barridos usados para acumular estadística

# Carpeta de salida: ../figuras respecto a este archivo
directorio_figuras = os.path.join(os.path.dirname(__file__), "..", "figuras")
os.makedirs(directorio_figuras, exist_ok=True)

# -----
# Acción euclidiana discretizada
# -----
def energia_total(camino, eps, omega, masa):
    """Acción euclidiana total del camino (red periódica).


$$S_E = \sum_i \left[ \frac{m}{2} (x_{i+1} - x_i)^2 / \text{eps} + \text{eps} \left( \frac{m \omega^2}{2} x_i^2 \right) \right].$$


    El primer término es la energía cinética discreta (diferencia de posiciones
```

Apéndice: código completo IV

*entre eslabones contiguos) y el segundo es la energía potencial del oscilador.
Se usa solo como diagnóstico; el muestreo emplea diferencias locales de acción.*

```
"""  
dx = np.roll(camino, -1) - camino          #  $x_{i+1} - x_i$  con frontera periódica  
cinetica = 0.5 * masa * np.sum(dx**2) / eps  
potencial = 0.5 * masa * omega**2 * eps * np.sum(camino**2)  
return cinetica + potencial
```

```
# -----  
# Un barrido de Metropolis (esquema rojo-negro, vectorizado)  
# -----
```

```
def barrido_metropolis(camino, eps, omega, masa, paso, hbar, rng):  
    """Actualiza una vez cada eslabón del camino con el criterio de Metropolis.
```

Para cada eslabón i se propone $x_i \rightarrow x_i + \delta$ (δ uniforme en $[-\text{paso}, \text{paso}]$) y se acepta el cambio si baja la acción local o, en caso contrario, con probabilidad $\exp(-\Delta S / \hbar)$. El parámetro $\hbar$ permite estudiar el efecto de permitir mayores fluctuaciones (ejercicio 5 del libro).

Los eslabones pares no son vecinos entre sí, de modo que se pueden actualizar todos a la vez; lo mismo ocurre con los impares. Esto vuelve cada barrido una operación vectorial de numpy en lugar de un bucle elemento a elemento.

```
"""  
n = len(camino)  
aceptados = 0  
for color in (0, 1):          # 0 = eslabones pares, 1 = impares
```

Apéndice: código completo V

```
idx = np.arange(color, n, 2)
x_viejo = camino[idx]
vecino_izq = camino[(idx - 1) % n]      # frontera periódica
vecino_der = camino[(idx + 1) % n]
# Desplazamiento propuesto para cada eslabón del color actual
x_nuevo = x_viejo + paso * (2.0 * rng.random(idx.size) - 1.0)
# Acción local (solo los términos que dependen del eslabón i)
cin_viejo = ((x_viejo - vecino_izq)**2 + (vecino_der - x_viejo)**2)
cin_nuevo = ((x_nuevo - vecino_izq)**2 + (vecino_der - x_nuevo)**2)
s_viejo = 0.5 * masa * cin_viejo / eps + 0.5 * masa * omega**2 * eps * x_viejo**2
s_nuevo = 0.5 * masa * cin_nuevo / eps + 0.5 * masa * omega**2 * eps * x_nuevo**2
delta_s = s_nuevo - s_viejo
# Criterio de Metropolis vectorizado
acepta = (delta_s <= 0.0) | (rng.random(idx.size) < np.exp(-delta_s / hbar))
camino[idx] = np.where(acepta, x_nuevo, x_viejo)
aceptados += int(np.count_nonzero(acepta))
return aceptados
```

```
# -----
# Simulación completa: termaliza, mide y acumula
# -----
```

```
def simular(hbar=1.0, n_instantaneas=0, bordes_hist=None, rng=None):
    """Ejecuta la cadena de Metropolis y devuelve la estadística acumulada.
```

Parámetros

Apéndice: código completo VI

hbar : valor efectivo de *hbar* en el peso de Boltzmann (1.0 = caso físico).
n_instantaneas : cuántos caminos completos guardar para graficar.
bordes_hist : bordes del histograma de posiciones; si es None se generan.
rng : generador aleatorio de numpy.

Devuelve un diccionario con el histograma de $|\psi_0|^2$, el valor de $\langle x^2 \rangle$, su historia acumulada (para ver la convergencia) y las instantáneas de caminos.
"""

```
if rng is None:
    rng = np.random.default_rng(semilla)
if bordes_hist is None:
    bordes_hist = np.linspace(-4.0, 4.0, 121)

camino = np.zeros(N_eslabones)           # camino inicial: todo en x = 0
centros = 0.5 * (bordes_hist[:-1] + bordes_hist[1:])
histograma = np.zeros(centros.size)

# Termalización: se descartan los primeros barridos
aceptados = 0
for _ in range(barridos_termalizacion):
    aceptados += barrido_metropolis(camino, eps, omega, masa, paso_propuesta, hbar, rng)
tasa_aceptacion = aceptados / (barridos_termalizacion * N_eslabones)

# Medición: se acumula el histograma y la media de  $x^2$ 
suma_x2 = 0.0
cuenta = 0
historia_x2 = np.zeros(barridos_medicion)
```

Apéndice: código completo VII

```
instantaneas = []
paso_guardado = max(1, barridos_medicion // max(1, n_instantaneas))

for k in range(barridos_medicion):
    barrido_metropolis(camino, eps, omega, masa, paso_propuesta, hbar, rng)
    conteo, _ = np.histogram(camino, bins=bordes_hist)
    histograma += conteo
    suma_x2 += np.sum(camino**2)
    cuenta += camino.size
    historia_x2[k] = suma_x2 / cuenta      # estimación acumulada de  $\langle x^2 \rangle$ 
    if n_instantaneas and (k % paso_guardado == 0) and len(instantaneas) < n_instantaneas:
        instantaneas.append(camino.copy())

# Normalización del histograma para que integre a 1 (densidad de probabilidad)
ancho = bordes_hist[1] - bordes_hist[0]
densidad = histograma / (np.sum(histograma) * ancho)

return {
    "centros": centros,
    "densidad": densidad,
    "x2_medio": historia_x2[-1],
    "historia_x2": historia_x2,
    "instantaneas": instantaneas,
    "tasa_aceptacion": tasa_aceptacion,
}
```

Apéndice: código completo VIII

```
# -----  
# Curva analítica de comparación: densidad del estado fundamental del oscilador  
# -----  
def densidad_analitica(x, hbar=1.0):  
    """ $|\psi_0(x)|^2 = \sqrt{m \omega / (\pi \hbar)} \exp(-m \omega x^2 / \hbar)$ ."""  
    coef = np.sqrt(masa * omega / (np.pi * hbar))  
    return coef * np.exp(-masa * omega * x**2 / hbar)  
  
# -----  
# Programa principal: ejecuta las simulaciones y genera las cuatro figuras  
# -----  
def main():  
    rng = np.random.default_rng(semilla)  
  
    # Simulación principal (caso físico  $\hbar = 1$ ), guardando algunas instantáneas  
    resultado = simular(hbar=1.0, n_instantaneas=6, rng=rng)  
    x = resultado["centros"]  
    print("Tasa de aceptación de Metropolis: {:.2f}".format(resultado["tasa_aceptacion"]))  
    print("Energía del estado fundamental estimada (virial): "  
          "E_0 =  $\omega^2 \langle x^2 \rangle = {:.4f}$ ".format(omega**2 * resultado["x2_medio"]))  
    print("Valor exacto: E_0 =  $\hbar \omega / 2 = {:.4f}$ ".format(0.5 * omega))  
    print("<math>x^2</math> estimado = {:.4f} (exacto = {:.4f)".format(resultado["x2_medio"], 0.5))  
  
    # Figura 1: caminos espacio-temporales muestreados  
    tau = np.arange(N_eslabones) * eps # tiempo imaginario de cada eslabón  
    fig1, ax1 = plt.subplots(figsize=(6, 7))
```

Apéndice: código completo IX

```
for i, camino in enumerate(resultado["instantaneas"]):
    ax1.plot(camino, tau, lw=1.0, alpha=0.7, label="Camino muestreado" if i == 0 else None)
ax1.axvline(0.0, color="black", lw=2.0, label="Trayectoria clásica (x = 0)")
ax1.set_xlabel("Posición x")
ax1.set_ylabel(r"Tiempo imaginario  $\tau$ ")
ax1.set_title("Caminos muestreados por Metropolis y trayectoria clásica")
ax1.grid(True, alpha=0.3)
ax1.legend(loc="upper right", fontsize=9)
fig1.tight_layout()
fig1.savefig(os.path.join(directorio_figuras, "caminos_espaciotemporales.pdf"))
plt.close(fig1)
```

```
# Figura 2: densidad del estado fundamental vs resultado analítico
fig2, ax2 = plt.subplots(figsize=(7, 5))
ax2.bar(x, resultado["densidad"], width=(x[1] - x[0]), alpha=0.55,
        color="tab:blue", label="Monte Carlo (Metropolis)")
xx = np.linspace(-4, 4, 400)
ax2.plot(xx, densidad_analitica(xx), color="tab:red", lw=2.0,
        label=r"Exacto:  $|\psi_0(x)|^2 = \pi^{-1/2} e^{-x^2}$ ")
ax2.set_xlabel("Posición x")
ax2.set_ylabel(r" $|\psi_0(x)|^2$ ")
ax2.set_title("Densidad del estado fundamental del oscilador armónico")
ax2.grid(True, alpha=0.3)
ax2.legend()
fig2.tight_layout()
fig2.savefig(os.path.join(directorio_figuras, "funcion_onda_fundamental.pdf"))
plt.close(fig2)
```

Apéndice: código completo X

```
# Figura 3: convergencia de la energía del estado fundamental
energia_acumulada = omega**2 * resultado["historia_x2"] # estimador del virial
barridos = np.arange(1, barridos_medicion + 1)
fig3, ax3 = plt.subplots(figsize=(7, 5))
ax3.plot(barridos, energia_acumulada, color="tab:blue", lw=1.2,
         label=r"Estimación del virial  $E_0 = \omega^2 \langle x^2 \rangle$ ")
ax3.axhline(0.5 * omega, color="tab:red", lw=2.0, ls="--",
           label=r"Valor exacto  $E_0 = \hbar\omega/2 = 0.5$ ")
ax3.set_xlabel("Barridos de medición")
ax3.set_ylabel(r" $E_0$  [unidades de  $\hbar\omega$ ]")
ax3.set_ylim(0.0, 1.0)
ax3.set_title("Convergencia de la energía del estado fundamental")
ax3.grid(True, alpha=0.3)
ax3.legend()
fig3.tight_layout()
fig3.savefig(os.path.join(directorio_figuras, "convergencia_energia.pdf"))
plt.close(fig3)

# Figura 4: efecto de hbar sobre las fluctuaciones (ejercicio 5)
fig4, ax4 = plt.subplots(figsize=(7, 5))
colores = {0.5: "tab:green", 1.0: "tab:blue", 2.0: "tab:purple"}
for hbar_ef in (0.5, 1.0, 2.0):
    res = simular(hbar=hbar_ef, rng=np.random.default_rng(semilla + int(10 * hbar_ef)))
    ax4.plot(res["centros"], res["densidad"], drawstyle="steps-mid",
            color=colores[hbar_ef], lw=1.6,
            label=r" $\hbar_{\{ef\}} = \{.1f\}$  ( $\langle x^2 \rangle = \{.2f\}$ )".format(
```

Apéndice: código completo XI

```
        hbar_ef, res["x2_medio"])))
ax4.set_xlabel("Posición x")
ax4.set_ylabel(r"$|\psi_0(x)|^2$")
ax4.set_title("Efecto de hbar: mayor hbar permite mayores fluctuaciones")
ax4.grid(True, alpha=0.3)
ax4.legend()
fig4.tight_layout()
fig4.savefig(os.path.join(directorio_figuras, "efecto_hbar.pdf"))
plt.close(fig4)

print("Figuras guardadas en:", os.path.normpath(directorio_figuras))

if __name__ == "__main__":
    main()
```